



Welcome to Python!

This Jupyter notebook contains a tutorial on Python. This tutorial should give you enough knowledge of Python to be able to have a basic ability to read and understand Python programs.

What is Python?

Python is a text-based programming language used by millions of professionals, data scientists, and web developers around the world. Unlike visual programming languages like SNAP! and Scratch, where you drag and drop blocks to create programs, Python requires you to **type actual code** using keywords and syntax. Python is known for being beginner-friendly because its syntax is relatively readable and looks similar to English, making it easier to understand and remember.

Key Differences Between Python and Visual Programming (SNAP! / Scratch):

Aspect	SNAP! / Scratch	Python
Input Method	Drag-and-drop visual blocks	Type code text
Learning Curve	Faster for absolute beginners	Slightly steeper but more powerful
Readability	Visual blocks are self-explanatory	Requires learning syntax and keywords
Real-World Use	Educational/learning tool	Professional industry standard
Projects	Games, animations, interactive stories	Web apps, data analysis, automation, AI

Aspect	SNAP! / Scratch	Python
Code Reusability	Limited to platform	Easily shared and used anywhere
Error Messages	More helpful and visual	More technical, requires debugging skills

Why Learn Python?

- **In-Demand Skill** — Used in tech companies, startups, and scientific research
- **Versatile** — Build websites, analyze data, create AI, automate tasks, and more
- **Transferable Knowledge** — Once you learn Python, picking up other languages is easier
- **Active Community** — Millions of programmers share code, libraries, and solutions online
- **Foundation for Advanced Programming** — A stepping stone to more complex programming concepts

The transition from visual programming to Python is like the difference between drawing pictures to tell a story versus writing a story with words—both work, but writing gives you more power and flexibility!

Executing Python Code

To run a Python code cell, simply click inside the cell and press **Shift + Enter** (or **Ctrl + Enter** if you don't want to move to the next cell). The code will execute immediately, and any output will appear directly below the cell. You can also press the Play button (adjacent to the cell in VSC; in the toolbar)

You can run cells in any order, so feel free to experiment and re-run cells as you learn! However, in normal development, you will execute cells in order since typically code cells have dependencies on previous cells because of function definitions and variable declarations.

References

- <https://jupyter-notebook.readthedocs.io/en/latest/notebook.html>
- <https://mybinder.readthedocs.io/en/latest/introduction.html>
- <https://docs.python.org/3/tutorial/index.html>
- <https://docs.python.org/3/tutorial/introduction.html>
- <https://daringfireball.net/projects/markdown/syntax>

Python objects, basic types, and variables

Everything in Python is an **object** and every object in Python has a **type**. Some of the basic types include:

- **int** (integer; a whole number with no decimal place)
 - 10
 - -3
- **float** (float; a number that has a decimal place)
 - 7.41
 - -0.006
- **str** (string; a sequence of characters enclosed in single quotes, double quotes, or triple quotes)
 - 'this is a string using single quotes'
 - "this is a string using double quotes"
 - '''this is a triple quoted string using single quotes'''
 - """this is a triple quoted string using double quotes"""
- **bool** (boolean; a binary value that is either true or false)
 - True
 - False
- **NoneType** (a special type representing the absence of a value)
 - None

In Python, a **variable** is a name you specify in your code that maps to a particular **object**, object **instance**, or value.

By defining variables, we can refer to things by names that make sense to us. Names for variables can only contain letters, underscores (`_`), or numbers (no spaces, dashes, or other characters). Variable names must start with a letter or underscore.

Basic operators

In Python, there are different types of **operators** (special symbols) that operate on different values. Some of the basic operators include:

- arithmetic operators
 - `+` (addition)
 - `-` (subtraction)
 - `*` (multiplication)
 - `/` (division)
 - `**` (exponent)
- assignment operators

- `=` (assign a value)
- `+=` (add and re-assign; increment)
- `-=` (subtract and re-assign; decrement)
- `*=` (multiply and re-assign)
- comparison operators (return either `True` or `False`)
 - `==` (equal to)
 - `!=` (not equal to)
 - `<` (less than)
 - `<=` (less than or equal to)
 - `>` (greater than)
 - `>=` (greater than or equal to)

When multiple operators are used in a single expression, **operator precedence** determines which parts of the expression are evaluated in which order. Operators with higher precedence are evaluated first (like PEMDAS in math). Operators with the same precedence are evaluated from left to right.

- `()` parentheses, for grouping
- `**` exponent
- `*`, `/` multiplication and division
- `+`, `-` addition and subtraction
- `==`, `!=`, `<`, `<=`, `>`, `>=` comparisons

See <https://docs.python.org/3/reference/expressions.html#operator-precedence>

Arithmetic Operators

```
In [1]: # Addition
result = 5 + 3
print(result) # Output: 8
```

8

```
In [2]: # Subtraction
result = 10 - 4
print(result) # Output: 6
```

6

```
In [3]: # Multiplication
result = 6 * 7
print(result) # Output: 42
```

42

```
In [4]: # Division (always returns a float)
result = 15 / 3
print(result) # Output: 5.
```

5.0

```
In [5]: # Floor Division (rounds down)
result = 17 // 5
print(result) # Output: 3

# Modulo (remainder)
result = 17 % 5
print(result) # Output: 2
```

3

2

```
In [6]: # Exponent (power)
result = 2 ** 3
print(result) # Output: 8
```

8

Comparison Operators

```
In [7]: # Equal to
print(5 == 5)      # True
print(5 == 3)      # False
```

True

False

```
In [8]: # Not equal to
print(5 != 3)      # True
print(5 != 5)      # False
```

True

False

```
In [9]: # Greater than
print(10 > 5)       # True
print(3 > 5)        # False
```

True

False

```
In [10]: # Less than
print(3 < 5)        # True
print(10 < 5)       # False
```

True

False

```
In [11]: # Greater than or equal
print(5 >= 5)       # True
print(3 >= 5)       # False
```

True

False

```
In [12]: # Less than or equal
print(3 <= 5)       # True
```

```
print(10 <= 5)    # False
```

True

False

```
In [13]: # AND operator (BOTH must be True)
print(True and True)    # True
print(True and False)   # False
print(5 > 3 and 3 > 1)   # True
```

True

False

True

```
In [14]: # OR operator (at Least ONE must be True)
print(True or False)    # True
print(False or False)   # False
print(5 > 10 or 3 > 1)   # True
```

True

False

True

```
In [15]: # NOT operator (reverses True/False)
print(not True)          # False
print(not False)         # True
print(not 5 > 10)        # True
```

False

True

True

Assignment Operators

```
In [16]: # Simple assignment
x = 10
x
```

Out[16]: 10

```
In [17]: # Add and assign
x += 5    # Same as: x = x + 5
print(x)  # Output: 15
```

15

```
In [18]: # Subtract and assign
x -= 3    # Same as: x = x - 3
print(x)  # Output: 12
```

12

```
In [19]: # Multiply and assign
x *= 2    # Same as: x = x * 2
print(x)  # Output: 24
```

24

```
In [20]: # Divide and assign
x /= 4      # Same as: x = x / 4
print(x)    # Output: 6.0
```

6.0

```
In [21]: # Example 1: Calculate total cost with tax
price = 50
tax_rate = 0.08
total = price + (price * tax_rate)
print(f"Total: ${total}") # Output: Total: $54.0
```

Total: \$54.0

```
In [22]: # Example 2: Check if age is valid for voting
age = 18
can_vote = age >= 18
print(can_vote) # Output: True
```

True

```
In [23]: # Example 3: Simple temperature converter
celsius = 25
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F") # Output: 25°C = 77.0°F
```

25°C = 77.0°F

Strings for Beginners

A **string** is a sequence of characters enclosed in quotes. Strings are one of the most common data types in Python and are used to store text data.

Creating Strings

You can create strings using:

- Single quotes: `'Hello'`
- Double quotes: `"Hello"`
- Triple quotes (for multi-line): `'''Hello World'''` or `"""Hello World"""`

```
In [24]: # Creating strings with different quote styles
string1 = 'Hello, World!'
string2 = "Python is fun"
string3 = '''This is a
multi-line string
using triple quotes'''

print(string1)
print(string2)
print(string3)

# Add /n to each print statement to separate the outputs for clarity.
```

```
Hello, World!  
Python is fun  
This is a  
multi-line string  
using triple quotes
```

```
In [25]: # String concatenation (joining strings with +)  
first_name = "Alice"  
last_name = "Smith"  
full_name = first_name + " " + last_name  
print(full_name) # Output: Alice Smith
```

Alice Smith

```
In [26]: # Another example  
greeting = "Hello, " + first_name + "!"  
print(greeting) # Output: Hello, Alice!
```

Hello, Alice!

```
In [27]: # String repetition (repeating strings with *)  
word = "Ha"  
laugh = word * 5  
print(laugh) # Output: HaHaHaHaHa
```

HaHaHaHaHa

```
In [28]: # Repeating a phrase  
separator = "-" * 10  
print(separator) # Output: -----  
print("Start" + separator + "End")
```

Start-----End

```
In [29]: # String Length (number of characters)  
text = "Python"  
length = len(text)  
print(f"The string '{text}' has {length} characters")  
# Output: The string 'Python' has 6 characters
```

The string 'Python' has 6 characters

```
In [30]: # Another example  
sentence = "Hello, World!"  
print(f"Length: {len(sentence)}") # Output: Length: 13
```

Length: 13

```
In [31]: # Accessing individual characters (indexing)  
# Remember: indexing starts at 0!  
text = "Python"  
  
first_char = text[0]  
print(f"First character: {first_char}") # Output: P  
  
second_char = text[1]  
print(f"Second character: {second_char}") # Output: y
```

```

last_char = text[-1]
print(f"Last character: {last_char}") # Output: n

# Print each character
print("\nAll characters:")
for char in text:
    print(char)

```

First character: P
 Second character: y
 Last character: n

All characters:
 P
 y
 t
 h
 o
 n

```

In [32]: # String slicing (getting a part of a string)
text = "Hello, World!"

# Get characters from index 0 to 4 (not including 5)
slice1 = text[0:5]
print(f"text[0:5] = '{slice1}'") # Output: Hello

# Get characters from index 7 onwards
slice2 = text[7:]
print(f"text[7:] = '{slice2}'") # Output: World!

# Get characters up to index 5
slice3 = text[:5]
print(f"text[:5] = '{slice3}'") # Output: Hello

# Every 2nd character
slice4 = text[::2]
print(f"text[::2] = '{slice4}'") # Output: Hlo ol!

```

text[0:5] = 'Hello'
 text[7:] = 'World!'
 text[:5] = 'Hello'
 text[::2] = 'Hlo ol!'

```

In [33]: # Common string methods for beginners
text = "  hello, world!  "

# Remove leading and trailing whitespace
stripped = text.strip()
print(f"Original: '{text}'")
print(f"Stripped: '{stripped}'")

```

Original: ' hello, world! '
 Stripped: 'hello, world!'

```

In [34]: # Convert to uppercase
upper = text.upper()

```

```
print(f"Uppercase: '{upper}')
```

Uppercase: ' HELLO, WORLD! '

```
In [35]: # Convert to Lowercase
lower = text.lower()
print(f"Lowercase: '{lower}')
```

Lowercase: ' hello, world! '

```
In [36]: # Replace a word
replaced = text.replace("world", "Python")
print(f"Replaced: '{replaced}')
```

Replaced: ' hello, Python! '

```
In [37]: # Check if string starts with something
print(f"Starts with 'hello': {text.strip().startswith('hello')}") # True
```

Starts with 'hello': True

```
In [38]: # Check if string ends with something
print(f"Ends with '!': {text.strip().endswith('!')}") # True
```

Ends with '!': True

```
In [39]: # String formatting with f-strings (modern way)
name = "Alice"
age = 25
city = "New York"

# Basic f-string
message = f"My name is {name}"
print(message) # Output: My name is Alice

# Multiple variables
sentence = f"{name} is {age} years old and lives in {city}"
print(sentence) # Output: Alice is 25 years old and lives in New York

# With expressions
next_year_age = age + 1
print(f"Next year, {name} will be {next_year_age}") # Output: Next year, Alice wil
```

My name is Alice

Alice is 25 years old and lives in New York

Next year, Alice will be 26

```
In [40]: # Formatting numbers
price = 19.99
quantity = 3
total = price * quantity
print(f"Total price: ${total:.2f}") # Output: Total price: $59.97
```

Total price: \$59.97

```
In [41]: # Practical string examples for beginners

# Example 1: Creating a greeting
user_name = input("What is your name? ") # This would prompt the user
```

```
# For now, let's use a predefined name
user_name = "Bob"
greeting = f"Welcome, {user_name}! Have a great day!"
print(greeting)
```

Welcome, Bob! Have a great day!

```
In [42]: # Example 2: Simple email validation check
email = "user@example.com"
has_at_sign = "@" in email
has_dot = "." in email
is_valid = has_at_sign and has_dot
print(f"Email '{email}' is valid: {is_valid}")
```

Email 'user@example.com' is valid: True

```
In [43]: # Example 3: Creating a formatted receipt
item = "Notebook"
price = 5.50
quantity = 3
print("\nReceipt:")
print("-" * 30)
print(f"Item: {item}")
print(f"Price: ${price:.2f}")
print(f"Quantity: {quantity}")
print(f"Total: ${price * quantity:.2f}")
print("-" * 30)
```

Receipt:

```
-----
Item: Notebook
Price: $5.50
Quantity: 3
Total: $16.50
-----
```

```
In [44]: # Example 4: Extracting information from a string
full_address = "123 Main Street, New York, NY 10001"
parts = full_address.split(", ")
street = parts[0]
city = parts[1]
print(f"\nYou live in {city} on {street}")
```

You live in New York on 123 Main Street

Key String Concepts Summary

Concept	Description	Example
Concatenation	Join strings with <code>+</code>	<code>"Hello" + " " + "World"</code>
Repetition	Repeat strings with <code>*</code>	<code>"Ha" * 3</code> → <code>"HaHaHa"</code>
Length	Get string length with <code>len()</code>	<code>len("Python")</code> → <code>6</code>
Indexing	Access character at index	<code>"Python"[0]</code> → <code>"P"</code>

Concept	Description	Example
Slicing	Get substring with <code>[start:end]</code>	<code>"Hello"[1:4]</code> → <code>"ell"</code>
Formatting	Insert variables with f-strings	<code>f"Hi {name}"</code>
Methods	Built-in string functions	<code>.upper()</code> , <code>.lower()</code> , <code>.strip()</code>

Important Notes:

- Strings are **immutable** — you cannot change them, only create new ones
- Indexing starts at **0**, not 1
- Negative indexing starts from the end: `-1` is the last character
- Strings are **case-sensitive**: `"Hello"` ≠ `"hello"`

Lists

Note: **mutable** objects can be modified after creation and **immutable** objects cannot.

- **list** (list: mutable; indexed by integers; items are stored in the order they were added)
 - `[3, 5, 6, 3, 'dog', 'cat', False]`

Strings and lists are **sequence types** that can use the `+`, `*`, `+=`, and `*=` operators.

```
In [45]: # Assign some containers to different variables
list1 = [3, 5, 6, 3, 'dog', 'cat', False]
```

```
In [46]: # Items in the list object are stored in the order they were added
list1
```

```
Out[46]: [3, 5, 6, 3, 'dog', 'cat', False]
```

```
In [47]: # Add and re-assign
list1 += [5, 'grapes']
list1
```

```
Out[47]: [3, 5, 6, 3, 'dog', 'cat', False, 5, 'grapes']
```

```
In [48]: # Multiply
[1, 2, 3, 4] * 2
```

```
Out[48]: [1, 2, 3, 4, 1, 2, 3, 4]
```

```
In [49]: # Multiply
(1, 2, 3, 4) * 3
```

```
Out[49]: (1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4)
```

Python built-in functions and callables

A **function** is a Python object that you can "call" to **perform an action** or compute and **return another object**. You call a function by placing parentheses to the right of the function name. Some functions allow you to pass **arguments** inside the parentheses (separating multiple arguments with a comma). Internal to the function, these arguments are treated like variables.

Python has several useful built-in functions to help you work with different objects and/or your environment. Here is a small sample of them:

- `type(obj)` to determine the type of an object
- `len(container)` to determine how many items are in a container
- `sorted(container)` to return a new list from a container, with the items sorted
- `sum(container)` to compute the sum of a container of numbers
- `min(container)` to determine the smallest item in a container
- `max(container)` to determine the largest item in a container
- `abs(number)` to determine the absolute value of a number

Complete list of built-in functions:

<https://docs.python.org/3/library/functions.html>

There are also different ways of defining your own functions and callable objects that we will explore later.

```
In [50]: simple_string1 = "Hello, World!"  
simple_string2 = "Python is fun"
```

```
In [51]: # Use the type() function to determine the type of an object  
type(simple_string1)
```

```
Out[51]: str
```

```
In [52]: # Use the len() function to determine how many items are in a container  
len(simple_string2)
```

```
Out[52]: 13
```

```
In [53]: # Use the sorted() function to return a new list from a container, with the items s  
sorted([10, 1, 3.6, 7, 5, 2, -3])
```

```
Out[53]: [-3, 1, 2, 3.6, 5, 7, 10]
```

```
In [54]: # Use the sorted() function to return a new list from a container, with the items s  
# - notice that capitalized strings come first  
sorted(['dogs', 'cats', 'zebras', 'Chicago', 'California', 'ants', 'mice'])
```

```
Out[54]: ['California', 'Chicago', 'ants', 'cats', 'dogs', 'mice', 'zebras']
```

```
In [55]: # Use the sum() function to compute the sum of a container of numbers  
sum([10, 1, 3.6, 7, 5, 2, -3])
```

```
Out[55]: 25.6
```

```
In [56]: # Use the min() function to determine the smallest item in a container  
min([10, 1, 3.6, 7, 5, 2, -3])
```

```
Out[56]: -3
```

```
In [57]: # Use the min() function to determine the smallest item in a container  
min(['g', 'z', 'a', 'y'])
```

```
Out[57]: 'a'
```

```
In [58]: # Use the max() function to determine the largest item in a container  
max([10, 1, 3.6, 7, 5, 2, -3])
```

```
Out[58]: 10
```

```
In [59]: # Use the max() function to determine the largest item in a container  
max('gibberish')  
# When you use the max() function on a string, it returns the character with the hi  
# has the highest value among the characters in 'gibberish'.
```

```
Out[59]: 's'
```

```
In [60]: # Use the abs() function to determine the absolute value of a number. The absolute  
# that number and zero on the number line. For example, the absolute value of -10 is  
abs(10)
```

```
Out[60]: 10
```

```
In [61]: # Use the abs() function to determine the absolute value of a number  
abs(-12)
```

```
Out[61]: 12
```

Some methods on string objects

As was previously stated, everything in Python is an object. Objects in Python have **methods**. Methods are **functions** that are associated with an object. In Python, strings are objects, and they have methods that allow you to perform various operations on them.

- **.capitalize()** to return a capitalized version of the string (only first char uppercase)
- **.upper()** to return an uppercase version of the string (all chars uppercase)
- **.lower()** to return a lowercase version of the string (all chars lowercase)
- **.count(substring)** to return the number of occurrences of the substring in the string

- `.startswith(substring)` to determine if the string starts with the substring
- `.endswith(substring)` to determine if the string ends with the substring
- `.replace(old, new)` to return a copy of the string with occurrences of the "old" replaced by "new"

```
In [62]: # Assign a string to a variable
a_string = 'tHis is a sTriNg'
```

```
In [63]: # Return a capitalized version of the string. capitalization means that the first c
# uppercase, and all other characters are converted to lowercase.
a_string.capitalize()
```

```
Out[63]: 'This is a string'
```

```
In [64]: # Return an uppercase version of the string
a_string.upper()
```

```
Out[64]: 'THIS IS A STRING'
```

```
In [65]: # Return a lowercase version of the string
a_string.lower()
```

```
Out[65]: 'this is a string'
```

```
In [66]: # Notice that the methods called have not actually modified the string
a_string
```

```
Out[66]: 'tHis is a sTriNg'
```

```
In [67]: # Count number of occurrences of a substring in the string
a_string.count('i')
```

```
Out[67]: 3
```

```
In [68]: # Count number of occurrences of a substring in the string after a certain position
a_string.count('i', 7)
```

```
Out[68]: 1
```

```
In [69]: # Count number of occurrences of a substring in the string
a_string.count('is')
```

```
Out[69]: 2
```

```
In [70]: # Does the string start with 'this'?
a_string.startswith('this')
```

```
Out[70]: False
```

```
In [71]: # Does the lowercase string start with 'this'?
a_string.lower().startswith('this')
```

Out[71]: True

```
In [72]: # Does the string end with 'Ng'?  
a_string.endswith('Ng')
```

Out[72]: True

```
In [73]: # Return a version of the string with a substring replaced with something else  
a_string.replace('is', 'XYZ')
```

Out[73]: 'tHXYZ XYZ a sTriNg'

```
In [74]: # Return a version of the string with a substring replaced with something else  
a_string.replace('i', '!')
```

Out[74]: 'tH!s !s a sTr!Ng'

```
In [75]: # Return a version of the string with the first 2 occurrences a substring replaced w  
a_string.replace('i', '!', 2)
```

Out[75]: 'tH!s !s a sTriNg'

Formatting strings

```
In [76]: name = "Alice"  
age = 25  
price = 19.99  
  
# Basic  
print(f"Hello, {name}!") # Hello, Alice!  
  
# With expressions  
print(f"{name} is {age + 1} next year") # Alice is 26 next year  
  
# Decimal places  
print(f"Price: ${price:.2f}") # Price: $19.99  
  
# Padding  
print(f"Score: {85:03d}") # Score: 085
```

Hello, Alice!
Alice is 26 next year
Price: \$19.99
Score: 085

```
In [77]: name = "Bob"  
age = 30  
  
# Positional  
print("Hello, {}".format(name)) # Hello, Bob!  
  
# Named  
print("Name: {name}, Age: {age}".format(name="Bob", age=30))
```

```
# With formatting
print("Price: ${:.2f}".format(19.99)) # Price: $19.99
```

Hello, Bob!

Name: Bob, Age: 30

Price: \$19.99

Python "for loops"

It is easy to **iterate** over a collection of items using a **for loop**. The strings, lists, tuples, sets, and dictionaries we defined are all **iterable** containers.

The for loop will go through the specified container, one item at a time, and provide a temporary variable for the current item. You can use this temporary variable like a normal variable.

```
In [78]: fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
# Output: apple, banana, cherry
```

apple

banana

cherry

```
In [80]: # Range from 0 to 4 (doesn't include 5)
for i in range(5):
    print(i)
# Output: 0, 1, 2, 3, 4
```

0

1

2

3

4

```
In [81]: # Range with start and end
for i in range(2, 6):
    print(i)
# Output: 2, 3, 4, 5
```

2

3

4

5

```
In [82]: # Range with step
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8
```

0
2
4
6
8

```
In [83]: word = "hello"

for letter in word:
    print(letter)
# Output: h, e, l, l, o
```

h
e
l
l
o

```
In [84]: # The % operator gives the remainder of a division operation
numbers = [1, 2, 3, 4, 5]

for num in numbers:
    if num % 2 == 0:
        print(f"{num} is even")
# Output: 2 is even, 4 is even
```

2 is even
4 is even

Python "if statements" and "while loops"

Conditional expressions can be used with these two **conditional statements**.

The **if statement** allows you to test a condition and perform some actions if the condition evaluates to `True`. You can also provide `elif` and/or `else` clauses to an if statement to take alternative actions if the condition evaluates to `False`.

The **while loop** will keep looping until its conditional expression evaluates to `False`.

Note: It is possible to "loop forever" when using a while loop with a conditional expression that never evaluates to `False`.

Note: Since the **for loop** will iterate over a container of items until there are no more, there is no need to specify a "stop looping" condition.

```
In [85]: age = 18

if age >= 18:
    print("You are an adult")
# Output: You are an adult

# If condition is false, nothing prints
age = 15
```

```
if age >= 18:
    print("You are an adult")
# Output: (nothing)
```

You are an adult

```
In [86]: temperature = 30

if temperature > 25:
    print("It's hot outside")
    print("Wear light clothes")
# Output:
# It's hot outside
# Wear light clothes
```

It's hot outside
Wear light clothes

```
In [87]: score = 75

if score >= 80:
    print("You passed!")
else:
    print("You failed!")
# Output: You failed!

# Another example
name = "Alice"
if name == "Bob":
    print("Hello Bob")
else:
    print("You are not Bob")
# Output: You are not Bob
```

You failed!
You are not Bob

```
In [88]: grade = 85

if grade >= 90:
    print("A")
elif grade >= 80:
    print("B")
elif grade >= 70:
    print("C")
else:
    print("F")
# Output: B
```

B

```
In [89]: age = 25
has_license = True

if age >= 18:
    if has_license:
        print("You can drive")
    else:
```

```

        print("You need a license to drive")
    else:
        print("You are too young to drive")
# Output: You can drive

# Another nested example
score = 85
if score >= 80:
    print("You passed!")
    if score >= 90:
        print("Great job!")
    else:
        print("Good effort")
# Output:
# You passed!
# Good effort

```

You can drive

You passed!

Good effort

In [90]: `x = 10`

```

if x == 10:      # equals
    print("x is 10")

if x != 5:       # not equal
    print("x is not 5")

if x > 8:        # greater than
    print("x is greater than 8")

if x < 20:       # less than
    print("x is less than 20")

if x >= 10:      # greater than or equal
    print("x is at least 10")

if x <= 10:      # less than or equal
    print("x is at most 10")

```

x is 10

x is not 5

x is greater than 8

x is less than 20

x is at least 10

x is at most 10

In [91]:

```

print("equals:", x == 10)
print("not equal:", x != 5)
print("greater than:", x > 8)
print("less than:", x < 20)
print("greater than or equal:", x >= 10)
print("less than or equal:", x <= 10)

```

```

equals: True
not equal: True
greater than: True
less than: True
greater than or equal: True
less than or equal: True

```

Lists for Beginners

A **list** is a collection of items stored in a single variable. Lists are **mutable** (you can change them) and **ordered** (items keep their position). Lists can contain items of different types.

Creating Lists

You create a list by placing items inside square brackets `[]`, separated by commas:

```

In [92]: # Creating lists with different types of items
numbers = [1, 2, 3, 4, 5]
print("Numbers:", numbers)

fruits = ["apple", "banana", "cherry", "date"]
print("Fruits:", fruits)

# Mixed data types in a single list
mixed = [42, "hello", 3.14, True, None]
print("Mixed:", mixed)

# Empty list
empty_list = []
print("Empty:", empty_list)

# List with repeated items
colors = ["red", "blue", "red", "green"]
print("Colors:", colors)

```

```

Numbers: [1, 2, 3, 4, 5]
Fruits: ['apple', 'banana', 'cherry', 'date']
Mixed: [42, 'hello', 3.14, True, None]
Empty: []
Colors: ['red', 'blue', 'red', 'green']

```

```

In [93]: # Accessing individual items in a list (indexing starts at 0)
fruits = ["apple", "banana", "cherry", "date"]

first_fruit = fruits[0]
print(f"First fruit: {first_fruit}") # Output: apple

second_fruit = fruits[1]
print(f"Second fruit: {second_fruit}") # Output: banana

# Negative indexing (from the end)
last_fruit = fruits[-1]
print(f"Last fruit: {last_fruit}") # Output: date

```

```
second_last_fruit = fruits[-2]
print(f"Second to last: {second_last_fruit}") # Output: cherry
```

First fruit: apple
 Second fruit: banana
 Last fruit: date
 Second to last: cherry

```
In [94]: # Slicing lists (getting a portion of a list)
numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90]

# Get items from index 1 to 4 (not including 4)
slice1 = numbers[1:4]
print(f"numbers[1:4] = {slice1}") # Output: [20, 30, 40]

# Get items from index 0 to 5
slice2 = numbers[0:5]
print(f"numbers[0:5] = {slice2}") # Output: [10, 20, 30, 40, 50]

# Get items from index 5 onwards
slice3 = numbers[5:]
print(f"numbers[5:] = {slice3}") # Output: [60, 70, 80, 90]

# Get first 3 items
slice4 = numbers[:3]
print(f"numbers[:3] = {slice4}") # Output: [10, 20, 30]

# Every 2nd item
slice5 = numbers[::2]
print(f"numbers[::2] = {slice5}") # Output: [10, 30, 50, 70, 90]

# Reverse the list
reversed_list = numbers[::-1]
print(f"Reversed: {reversed_list}")
```

```
numbers[1:4] = [20, 30, 40]
numbers[0:5] = [10, 20, 30, 40, 50]
numbers[5:] = [60, 70, 80, 90]
numbers[:3] = [10, 20, 30]
numbers[::2] = [10, 30, 50, 70, 90]
Reversed: [90, 80, 70, 60, 50, 40, 30, 20, 10]
```

```
In [95]: # Modifying Lists - Changing items
shopping_list = ["milk", "eggs", "bread", "butter"]
print(f"Original: {shopping_list}")

# Change an item
shopping_list[1] = "cheese"
print(f"After change: {shopping_list}")

# Change multiple items
shopping_list[0:2] = ["yogurt", "bacon"]
print(f"After slice change: {shopping_list}")
```

```
Original: ['milk', 'eggs', 'bread', 'butter']
After change: ['milk', 'cheese', 'bread', 'butter']
After slice change: ['yogurt', 'bacon', 'bread', 'butter']
```

```
In [96]: # Adding items to a list
# Method 1: append() - adds one item to the end
fruits = ["apple", "banana"]
print(f"Original: {fruits}")

fruits.append("cherry")
print(f"After append: {fruits}") # Output: ['apple', 'banana', 'cherry']

# Method 2: insert() - adds item at a specific index
fruits.insert(1, "orange")
print(f"After insert at 1: {fruits}") # Output: ['apple', 'orange', 'banana', 'che']

# Method 3: extend() - adds multiple items from another list
more_fruits = ["date", "fig"]
fruits.extend(more_fruits)
print(f"After extend: {fruits}") # Output: ['apple', 'orange', 'banana', 'cherry', 'date', 'fig']

# Using += operator (also extends the list)
vegetables = ["carrot", "lettuce"]
vegetables += ["broccoli", "spinach"]
print(f"Vegetables: {vegetables}")
```

```
Original: ['apple', 'banana']
After append: ['apple', 'banana', 'cherry']
After insert at 1: ['apple', 'orange', 'banana', 'cherry']
After extend: ['apple', 'orange', 'banana', 'cherry', 'date', 'fig']
Vegetables: ['carrot', 'lettuce', 'broccoli', 'spinach']
```

```
In [97]: # Removing items from a list
# Method 1: remove() - removes the first occurrence of a value
items = ["apple", "banana", "cherry", "banana", "date"]
print(f"Original: {items}")

items.remove("banana")
print(f"After remove('banana'): {items}") # Removes first "banana"

# Method 2: pop() - removes item at specific index (returns the item)
removed_item = items.pop(0)
print(f"After pop(0), removed: {removed_item}")
print(f"List now: {items}")

# pop() without index removes the last item
last_item = items.pop()
print(f"After pop(), removed: {last_item}")
print(f"List now: {items}")

# Method 3: del statement - deletes item at index or slice
colors = ["red", "blue", "green", "yellow", "purple"]
del colors[1]
print(f"After del colors[1]: {colors}")

# Delete a range
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[2:5]
print(f"After del numbers[2:5]: {numbers}")
```

```
# Method 4: clear() - removes all items
temp_list = [1, 2, 3, 4, 5]
temp_list.clear()
print(f"After clear(): {temp_list}")
```

Original: ['apple', 'banana', 'cherry', 'banana', 'date']
 After remove('banana'): ['apple', 'cherry', 'banana', 'date']
 After pop(0), removed: apple
 List now: ['cherry', 'banana', 'date']
 After pop(), removed: date
 List now: ['cherry', 'banana']
 After del colors[1]: ['red', 'green', 'yellow', 'purple']
 After del numbers[2:5]: [1, 2, 6, 7, 8, 9]
 After clear(): []

```
In [98]: # Useful List methods and operations
numbers = [3, 1, 4, 1, 5, 9, 2, 6]

# len() - get the length (number of items)
print(f"Length: {len(numbers)}") # Output: 8

# count() - count occurrences of a value
count_ones = numbers.count(1)
print(f"Count of 1: {count_ones}") # Output: 2

# index() - find the index of first occurrence
index_five = numbers.index(5)
print(f"Index of 5: {index_five}") # Output: 4

# sort() - sorts the list in place (modifies the original)
to_sort = [3, 1, 4, 1, 5, 9, 2, 6]
to_sort.sort()
print(f"Sorted: {to_sort}") # Output: [1, 1, 2, 3, 4, 5, 6, 9]

# reverse() - reverses the list in place
to_reverse = [1, 2, 3, 4, 5]
to_reverse.reverse()
print(f"Reversed: {to_reverse}") # Output: [5, 4, 3, 2, 1]

# Check if item is in list
fruits = ["apple", "banana", "cherry"]
print(f"'apple' in list: {'apple' in fruits}") # True
print(f"'orange' in list: {'orange' in fruits}") # False

# copy() - creates a copy of the list
original = [1, 2, 3]
copy = original.copy()
copy.append(4)
print(f"Original: {original}") # [1, 2, 3]
print(f"Copy: {copy}") # [1, 2, 3, 4]
```

Length: 8
 Count of 1: 2
 Index of 5: 4
 Sorted: [1, 1, 2, 3, 4, 5, 6, 9]
 Reversed: [5, 4, 3, 2, 1]
 'apple' in list: True
 'orange' in list: False
 Original: [1, 2, 3]
 Copy: [1, 2, 3, 4]

```

In [99]: # Looping through lists
# Method 1: Simple for loop
fruits = ["apple", "banana", "cherry"]
print("Fruits:")
for fruit in fruits:
    print(f" - {fruit}")

# Method 2: Loop with index using enumerate()
print("\nWith index:")
for index, fruit in enumerate(fruits):
    print(f" {index}: {fruit}")

# Method 3: Loop using range and index
print("\nUsing range and index:")
for i in range(len(fruits)):
    print(f" Position {i} has {fruits[i]}")

# Example: Process each item
numbers = [1, 2, 3, 4, 5]
squares = []
for num in numbers:
    squares.append(num ** 2)
print(f"\nOriginal: {numbers}")
print(f"Squares: {squares}")
  
```

Fruits:

- apple
- banana
- cherry

With index:

- 0: apple
- 1: banana
- 2: cherry

Using range and index:

- Position 0 has apple
- Position 1 has banana
- Position 2 has cherry

Original: [1, 2, 3, 4, 5]

Squares: [1, 4, 9, 16, 25]

```

In [100... # Practical List examples

# Example 1: Grade tracking
  
```

```

grades = [85, 92, 78, 95, 88]
average = sum(grades) / len(grades)
print(f"Grades: {grades}")
print(f"Average: {average:.2f}")
print(f"Highest: {max(grades)}")
print(f"Lowest: {min(grades)}")

# Example 2: To-do List
print("\n--- To-Do List ---")
todo = []

# Add tasks
todo.append("Buy groceries")
todo.append("Finish homework")
todo.append("Call mom")
print(f"Tasks: {todo}")

# Mark first task as done (remove it)
completed = todo.pop(0)
print(f"Completed: {completed}")
print(f"Remaining: {todo}")

# Example 3: Filtering data
print("\n--- Even Numbers ---")
all_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = []
for num in all_numbers:
    if num % 2 == 0:
        even_numbers.append(num)
print(f"All: {all_numbers}")
print(f"Even: {even_numbers}")

# Example 4: Word frequency counter
text = "hello world hello python hello"
words = text.split()
print(f"\nWords: {words}")
print(f"'hello' appears {words.count('hello')} times")
print(f"'world' appears {words.count('world')} times")

```

Grades: [85, 92, 78, 95, 88]
 Average: 87.60
 Highest: 95
 Lowest: 78

--- To-Do List ---
 Tasks: ['Buy groceries', 'Finish homework', 'Call mom']
 Completed: Buy groceries
 Remaining: ['Finish homework', 'Call mom']

--- Even Numbers ---
 All: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
 Even: [2, 4, 6, 8, 10]

Words: ['hello', 'world', 'hello', 'python', 'hello']
 'hello' appears 3 times
 'world' appears 1 times

Key List Concepts Summary

Operation	Method	Description	Example
Create	<code>[]</code>	Create an empty list or with items	<code>[1, 2, 3]</code>
Access	<code>list[0]</code>	Get item at index	<code>fruits[0]</code> → first item
Slice	<code>list[1:4]</code>	Get items from index 1 to 3	<code>[1, 2, 3, 4, 5][1:4]</code> → <code>[2, 3, 4]</code>
Add	<code>.append()</code>	Add single item to end	<code>list.append(5)</code>
Add	<code>.insert()</code>	Add item at specific index	<code>list.insert(1, 5)</code>
Add	<code>.extend()</code>	Add multiple items	<code>list.extend([5, 6, 7])</code>
Remove	<code>.remove()</code>	Remove first occurrence	<code>list.remove(5)</code>
Remove	<code>.pop()</code>	Remove and return item	<code>list.pop()</code>
Remove	<code>.clear()</code>	Remove all items	<code>list.clear()</code>
Length	<code>len()</code>	Get number of items	<code>len([1, 2, 3])</code> → 3
Sort	<code>.sort()</code>	Sort in place	<code>list.sort()</code>
Reverse	<code>.reverse()</code>	Reverse in place	<code>list.reverse()</code>
Find	<code>.index()</code>	Find first occurrence	<code>list.index(5)</code>
Count	<code>.count()</code>	Count occurrences	<code>list.count(5)</code>
Check	<code>in</code>	Check if item exists	<code>5 in [1, 2, 5]</code> → <code>True</code>

Important Notes:

- Lists are **mutable** — you can change them after creation
- Indexing starts at **0**, not 1
- Negative indexing works: `-1` is the last item, `-2` is second to last
- Lists are **ordered** — items maintain their position
- Lists can contain **any data type** — numbers, strings, booleans, None, even other lists
- When you assign a list to a variable, changes affect the original: use `.copy()` to avoid this

Functions

A **function** is a reusable block of code that performs a specific task. Functions help you organize your code, avoid repetition, and make your programs easier to read and maintain. You define a function once using the `def` keyword, and then you can call it (use it) as many

times as you need. Functions can take **parameters** (inputs) and return **values** (outputs). Think of a function like a recipe: you write down the steps once, and whenever you want to make that dish, you just follow the recipe instead of figuring out all the steps again.

```
In [101... def greet():  
    print("Hello!")  
  
greet()  
# Output: Hello!
```

Hello!

```
In [102... def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")  
# Output: Hello, Alice!  
  
greet("Bob")  
# Output: Hello, Bob!
```

Hello, Alice!

Hello, Bob!

```
In [103... def add(a, b):  
    result = a + b  
    return result  
  
sum_result = add(5, 3)  
print(sum_result)  
# Output: 8  
  
print(add(10, 20))  
# Output: 30
```

8

30

```
In [104... def describe_person(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}")  
  
describe_person("Alice", 25, "New York")  
# Output: Alice is 25 years old and lives in New York  
  
describe_person("Bob", 30, "Boston")  
# Output: Bob is 30 years old and lives in Boston
```

Alice is 25 years old and lives in New York

Bob is 30 years old and lives in Boston

```
In [105... def greet(name, greeting="Hello"):  
    print(f"{greeting}, {name}!")  
  
greet("Alice")  
# Output: Hello, Alice!
```

```
greet("Bob", "Hi")  
# Output: Hi, Bob!  
  
greet("Charlie", "Hey")  
# Output: Hey, Charlie!
```

Hello, Alice!
Hi, Bob!
Hey, Charlie!

```
In [106... def get_user_info():  
    name = "Alice"  
    age = 25  
    return name, age  
  
name, age = get_user_info()  
print(f"{name} is {age}")  
# Output: Alice is 25
```

Alice is 25

```
In [107... def calculate_area(radius):  
    area = 3.14159 * radius * radius  
    return area  
  
result = calculate_area(5)  
print(f"Area: {result:.2f}")  
# Output: Area: 78.54
```

Area: 78.54

```
In [108... def check_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    else:  
        return "F"  
  
print(check_grade(85))  
# Output: B  
  
print(check_grade(95))  
# Output: A
```

B
A

```
In [109... def sum_numbers(numbers):  
    total = 0  
    for num in numbers:  
        total += num  
    return total  
  
result = sum_numbers([1, 2, 3, 4, 5])
```

```
print(result)
# Output: 15
```

15

In [110...

```
def square(x):
    return x * x

def sum_of_squares(a, b):
    return square(a) + square(b)

result = sum_of_squares(3, 4)
print(result)
# Output: 25 (9 + 16)
```

25

Key Points:

- **def** — keyword to define a function
- **Function name** — should be descriptive
- **Parameters** — variables inside the parentheses
- **Indentation** — code inside function must be indented
- **return** — sends value back to where function was called
- **Call function** — use function name with parentheses: `function_name()`

NumPy Library

NumPy is a Python library used for working with numbers and arrays. It is especially helpful when you need to do math quickly on many values at once.

A NumPy array is like a list, but it is designed for numerical calculations. It is faster and easier to use for scientific computing such as machine learning.

In [112...

```
%pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\dgodd\appdata\local\programs\python\python311\lib\site-packages (2.4.6)
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 24.0 -> 26.1.2
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [111...

```
import numpy as np
```

Why NumPy instead of plain Python lists?

Python lists are flexible, but they are slow for math on large amounts of data, because Python checks the type of every item, one at a time, as it loops through the list.

NumPy arrays store data in a single, fixed-type block of memory and perform operations on the whole array at once (this is called **vectorization**). This makes NumPy dramatically faster for numeric work, and it's the reason nearly every machine learning and data science library in Python is built on top of it.

In [113...

```
import time

# Compare a plain Python list vs. a NumPy array for the same task:
# squaring one million numbers

size = 1_000_000
python_list = list(range(size))
numpy_array = np.arange(size)

# Python list: Loop through every item one at a time
start = time.time()
squared_list = [x ** 2 for x in python_list]
list_time = time.time() - start

# NumPy array: vectorized operation on the whole array at once
start = time.time()
squared_array = numpy_array ** 2
numpy_time = time.time() - start

print(f"Python list time: {list_time:.4f} seconds")
print(f"NumPy array time: {numpy_time:.4f} seconds")
print(f"NumPy was about {list_time / numpy_time:.0f}x faster")
```

Python list time: 0.1440 seconds
 NumPy array time: 0.0313 seconds
 NumPy was about 5x faster

Create a NumPy array

In [114...

```
numbers = np.array([1, 2, 3, 4, 5])

print("Array:", numbers)
print("Type:", type(numbers))
```

Array: [1 2 3 4 5]
 Type: <class 'numpy.ndarray'>

Perform math on the array

In [115...

```
print("Add 2:", numbers + 2)
print("Multiply by 3:", numbers * 3)
print("Average:", np.mean(numbers))
```

Add 2: [3 4 5 6 7]
 Multiply by 3: [3 6 9 12 15]
 Average: 3.0

Other common ways to create arrays

You won't always type out every value by hand. NumPy has built-in functions for creating arrays of a given size or pattern.

```
In [116... zeros = np.zeros(5)           # array of 5 zeros
ones = np.ones((2, 3))        # 2x3 array of ones
sequence = np.arange(0, 10, 2) # 0, 2, 4, 6, 8 (start, stop, step)
evenly_spaced = np.linspace(0, 1, 5) # 5 evenly spaced numbers from 0 to 1

print("Zeros:", zeros)
print("Ones:\n", ones)
print("Arange:", sequence)
print("Linspace:", evenly_spaced)
```

```
Zeros: [0. 0. 0. 0. 0.]
Ones:
[[1. 1. 1.]
 [1. 1. 1.]]
Arange: [0 2 4 6 8]
Linspace: [0.    0.25 0.5   0.75 1.   ]
```

Indexing, slicing, and array info

Just like lists, arrays support indexing and slicing. Arrays also carry extra information about themselves, like their `shape` (dimensions) and `dtype` (the type of data stored inside).

```
In [117... numbers = np.array([10, 20, 30, 40, 50])

print("First item:", numbers[0])
print("Last item:", numbers[-1])
print("First three:", numbers[:3])
print("Shape:", numbers.shape) # how many elements, per dimension
print("Data type:", numbers.dtype) # what kind of numbers are stored
```

```
First item: 10
Last item: 50
First three: [10 20 30]
Shape: (5,)
Data type: int64
```

2D Arrays with NumPy

A 2D array is like a table with rows and columns. It is useful when you want to store and work with data in a grid.

NumPy makes it easy to create and manipulate 2D arrays.

Create a 2D array

```
In [118... matrix = np.array([[1, 2, 3],
                       [4, 5, 6]])
```

```
print("Matrix:")
print(matrix)
```

```
Matrix:
[[1 2 3]
 [4 5 6]]
```

Reshaping and combining arrays

`reshape()` lets you change an array's dimensions without changing its data. This matters a lot in machine learning, where data often needs to be reorganized into a specific shape before it can be fed into a model.

```
In [119... flat = np.arange(1, 7)      # [1, 2, 3, 4, 5, 6]
grid = flat.reshape(2, 3)    # reorganize into 2 rows, 3 columns

print("Flat array:", flat)
print("Reshaped into 2x3:")
print(grid)
```

```
Flat array: [1 2 3 4 5 6]
Reshaped into 2x3:
[[1 2 3]
 [4 5 6]]
```

Matrix multiplication (the dot product)

Multiplying two matrices together with `@` or `np.dot()` is not the same as multiplying them element-by-element. This operation, called the **dot product**, is the single most common calculation in machine learning: it's how a neural network combines inputs and weights at every layer.

```
In [ ]: A = np.array([[1, 2],
                     [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

print("Element-wise multiply (A * B):")
print(A * B)
```

```
Element-wise multiply (A * B):
[[ 5 12]
 [21 32]]
Matrix multiply / dot product (A @ B):
[[19 22]
 [43 50]]
```

$$\begin{array}{ll}
 A[0][0] * B[0][0] = 1 * 5 = 5 & A[0][1] * B[0][1] = 2 * 6 = 12 \\
 A[1][0] * B[1][0] = 3 * 7 = 21 & A[1][1] * B[1][1] = 4 * 8 = 32
 \end{array}$$

In [121...

```
print("Matrix multiply / dot product (A @ B):")
print(A @ B)
```

Matrix multiply / dot product (A @ B):

```
[[19 22]
 [43 50]]
```

```
result[0][0] = (A row 0) · (B col 0) = (1*5) + (2*7) = 5 + 14 = 19
result[0][1] = (A row 0) · (B col 1) = (1*6) + (2*8) = 6 + 16 = 22
result[1][0] = (A row 1) · (B col 0) = (3*5) + (4*7) = 15 + 28 = 43
result[1][1] = (A row 1) · (B col 1) = (3*6) + (4*8) = 18 + 32 = 50
```

Broadcasting

NumPy can automatically apply an operation between arrays of different shapes, as long as the shapes are compatible. This is called **broadcasting**, and it's what lets you write `numbers + 2` earlier in this notebook without writing a loop — NumPy "stretches" the smaller value across the whole array.

In [122...

```
matrix = np.array([[1, 2, 3],
                  [4, 5, 6]])

row_adjustment = np.array([10, 20, 30]) # shape (3,)

print("Matrix:")
print(matrix)
print("Matrix + row_adjustment (broadcast across each row):")
print(matrix + row_adjustment)
```

Matrix:

```
[[1 2 3]
 [4 5 6]]
```

Matrix + row_adjustment (broadcast across each row):

```
[[11 22 33]
 [14 25 36]]
```

Data Frames

Data frames are like spreadsheets in code: rows + columns with labels.

A DataFrame is actually built on top of NumPy arrays under the hood — it takes the same fast numeric arrays from the last section and adds row/column labels, mixed data types, and spreadsheet-like tools on top.

DataFrame vs tuple vs dictionary

This is a useful comparison for clarifying data structures. We're comparing:

- `pandas.DataFrame` (tabular, 2D)
 - `tuple` (ordered, immutable, 1D)
 - `dict` (unordered key-value mapping, 1D)
-

1) Shape and dimensions

- `tuple` : ordered sequence of items.
 - Example: `("Alice", 30, "NY")`
 - 1D, fixed length, index-based.
 - `dict` : mapping keys → values.
 - Example: `{"name": "Alice", "age": 30, "city": "NY"}`
 - 1D, no inherent row/column structure.
 - `DataFrame` : table (rows × columns).
 - Example:
 - `Name | Age | City`
 - `Alice | 30 | NY`
 - `Bob | 27 | LA`
 - 2D, can have many rows and columns.
-

2) Mutability and operations

- `tuple` : immutable (cannot change after create). Good for fixed records.
 - `dict` : mutable (update keys/values easily).
 - `DataFrame` : mutable and optimized for vectorized data operations (slicing, groupby, stats, join, missing data, time series, etc.).
-

3) Indexing style

- `tuple` : `t[0]`, `t[1]` (by position only).
 - `dict` : `d["key"]` (by key).
 - `DataFrame` :
 - column: `df["age"]` or `df.age`
 - row: `df.loc[row_label]`, `df.iloc[row_index]`
 - cell: `df.loc[row_label, "age"]`
-

4) Typical use

- `tuple` : small fixed records, simple data transfer.
- `dict` : fast lookup by key, config data, serialized objects.

- `DataFrame` : analytics, data cleaning, machine learning, multiple row records with columns.
-

5) Performance and scaling

- `tuple` : tiny, simple, low overhead.
 - `dict` : moderate, $O(1)$ lookup per key.
 - `DataFrame` : heavier but efficient for large tabular data (millions of rows), with built-in optimized numeric operations (C/Cython backend).
-

6) Example mapping

`tuple` row: ("Alice", 88, 92)

`dict` row: {"Name": "Alice", "Math": 88, "English": 92}

`DataFrame` : many rows + operations:

- `df.mean()` , `df.groupby("Name").sum()` , `df.to_csv()` , etc.
-

Quick takeaway

`DataFrame` = structured table engine.

`Tuple` = static ordered record.

`Dict` = key-value store.

Use `DataFrame` for real datasets; `tuple`/`dict` for lightweight single records or simple lookups.

Examples

1) Simple student score table

In [123...

```
import pandas as pd

df_students = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie"],
    "Math": [88, 76, 95],
    "English": [92, 81, 87],
    "Passed": [True, True, True]
})

print(df_students)
```

	Name	Math	English	Passed
0	Alice	88	92	True
1	Bob	76	81	True
2	Charlie	95	87	True

Result:

- Columns: Name , Math , English , Passed
 - Rows: one per student
-

2) Product inventory

```
In [124... df_inventory = pd.DataFrame([
    {"Item": "Pen", "Qty": 120, "Price": 1.20},
    {"Item": "Notebook", "Qty": 50, "Price": 2.75},
    {"Item": "Eraser", "Qty": 200, "Price": 0.60}
])

print(df_inventory)
```

	Item	Qty	Price
0	Pen	120	1.20
1	Notebook	50	2.75
2	Eraser	200	0.60

- Good for filter , sort , sum (e.g., total stock value)
-

3) Time series sales

```
In [125... df_sales = pd.DataFrame({
    "Date": pd.to_datetime(["2026-03-01", "2026-03-02", "2026-03-03"]),
    "Sales": [100, 140, 90]
})

print(df_sales)
```

	Date	Sales
0	2026-03-01	100
1	2026-03-02	140
2	2026-03-03	90

- Nice to see date-indexed data, rolling averages, plots.
-

4) Mixed types + missing values

```
In [126... df_mixed = pd.DataFrame({
    "City": ["Rome", "Nairobi", "Tokyo", None],
    "Population": [2_873_000, 4_397_000, 13_960_000, 8_336_000],
    "HasMetro": [True, False, True, True]
```

```
})

print(df_mixed)
```

	City	Population	HasMetro
0	Rome	2873000	True
1	Nairobi	4397000	False
2	Tokyo	13960000	True
3	NaN	8336000	True

- Shows NaN handling, dtype inference, fill/mask operations.

Common beginner operations

- `df.head()` — first rows
- `df.columns` — column labels
- `df.describe()` — stats for numeric cols
- `df.loc[0]` / `df.iloc[1]` — row selection
- `df[df["Sales"] > 100]` — filtering

Quick tips

- Use `pd.read_csv("file.csv")` to load.
- Use `df.to_csv("out.csv", index=False)` to save.
- DataFrame = core of pandas, best for tabular analytics.

From Arrays and DataFrames to Machine Learning

Everything in this notebook has been building toward one idea: **machine learning models don't understand spreadsheets, sentences, or images — they only understand numbers organized into arrays and matrices.**

Here's how the pieces fit together:

- **DataFrames** are how we store and clean real-world data — labeled rows and columns, mixed types, missing values handled gracefully. They're built for *people* to read and reason about.
- **NumPy arrays** are how that same data gets represented once it's ready for math — a matrix of pure numbers, with no labels, optimized for speed. They're built for *computers* to calculate with.
- **Machine learning models** (in libraries like scikit-learn, PyTorch, and TensorFlow) expect their input as NumPy arrays or matrices, not DataFrames. A DataFrame's `.to_numpy()`

method (or `.values`) is often the very last step before training a model.

- The **dot product** you just saw with `@` is not just a math curiosity — it is literally the core operation inside a neural network. Every layer of a neural network takes an array of inputs, multiplies it by a matrix of weights using a dot product, and passes the result to the next layer. What looks like "intelligence" from the outside is, underneath, this same operation from this notebook, repeated billions of times.

A small preview

Below, a DataFrame of student scores is converted into a NumPy array — the same kind of transformation that happens right before any dataset is used to train a model.

In [127... `%pip install pandas`

```
Requirement already satisfied: pandas in c:\users\dgodd\appdata\local\programs\python\python311\lib\site-packages (3.0.5)
Requirement already satisfied: numpy>=1.26.0 in c:\users\dgodd\appdata\local\programs\python\python311\lib\site-packages (from pandas) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\dgodd\appdata\roaming\python\python311\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\dgodd\appdata\roaming\python\python311\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in c:\users\dgodd\appdata\roaming\python\python311\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 24.0 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [128... `import pandas as pd`

```
df_preview = pd.DataFrame({
    "Math": [88, 76, 95],
    "English": [92, 81, 87]
})

print("As a DataFrame (readable, labeled):")
print(df_preview)

features = df_preview.to_numpy()
print("\nAs a NumPy array (what a machine learning model actually sees):")
print(features)
print("Shape:", features.shape) # (3 students, 2 features each)
```

As a DataFrame (readable, labeled):

	Math	English
0	88	92
1	76	81
2	95	87

As a NumPy array (what a machine learning model actually sees):

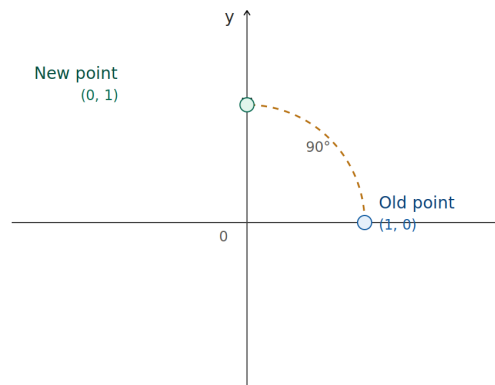
```
[[88 92]
 [76 81]
 [95 87]]
Shape: (3, 2)
```

Vector Arithmetic in Video Games

What's a simple example about how matrix math is done in video games?

A classic, simple one: **rotating an object on screen.**

Say you have a game character represented as a point at position (x, y) , and you want to rotate it around the origin by some angle. Graphics engines do this with a **rotation matrix** multiplied against the point's coordinates — same "row times column, multiply and add" rule as $A @ B$.



Rotating (1, 0) by 90° counter-clockwise gives (0, 1)

****The rotation matrix**** for an angle θ looks like this:

$$R = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

To rotate a point, you multiply this matrix by the point (written as a column):

$$\text{new_position} = R @ \text{point}$$

Concrete example — rotate the point (1, 0) by 90°:

$$R = \begin{pmatrix} \cos(90^\circ) & -\sin(90^\circ) \\ \sin(90^\circ) & \cos(90^\circ) \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

```
point = 1
      0
```

```
new_position[0] = (0 * 1) + (-1 * 0) = 0
new_position[1] = (1 * 1) + (0 * 0) = 1
```

Result: (1, 0) becomes (0, 1) — the point swung a quarter turn counter-clockwise, exactly what you'd expect from a 90° rotation.

```
In [ ]: import numpy as np

theta = np.pi / 2 # 90 degrees in radians
print(f"Rotating a point by {theta} radians (90 degrees)")

R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])

# A rotation matrix R is a 2x2 matrix that, when multiplied by a point (vector), rotates it by the
# specified angle theta. The matrix is defined as follows:
print("Rotation matrix R:")
print(R)

point = np.array([1, 0]) # current position of the point (1, 0) in 2D space

new_position = R @ point
print("Rotated point:", new_position)
```

```
Rotating a point by 1.5707963267948966 radians (90 degrees)
Rotation matrix R:
[[ 6.123234e-17 -1.000000e+00]
 [ 1.000000e+00  6.123234e-17]]
Rotated point: [6.123234e-17 1.000000e+00]
```

Why this connects to the bigger picture

A game engine does this same multiplication for **every** vertex of **every** character, for **every** frame — 60 times a second; thousands of points at once. That's exactly why NumPy's speed advantage from earlier in the notebook matters: doing this with Python loops instead of vectorized matrix math would make a game unplayably slow. Real engines also stack multiple transformations (move, then rotate, then scale) by multiplying several matrices together in sequence, which is another reason `@` (not `*`) is the operation that matters here (the dot product vs. element-wise multiplication).